

Dr. SPM International Institute of Information and Technology

Department of Data Science and Artificial Intelligence and Computer Science & Engineering

B.Tech Programs: Computer Science and Engineering (CSE) / Data Science and Artificial Intelligence (DSAI)

B.TECH MINOR PROJECT PROPOSAL

Project Title

Chhattisgarhi Voice-Based Agricultural Shopping Assistant

A Voice-First Agricultural Product Discovery and Cart-Management Prototype for Chhattisgarhi Speakers

Project Type	B.Tech Minor Project
Academic Session	Autumn 2026
Project Duration	Current academic project period to approximately mid-December 2026 (~16 weeks)
Student 1	Punyansh Thakur (Roll No. 241020461)
Student 2	Harsh Dadsena (Roll No. 241020433)
Student 3	Aakash Sen (Roll No. 241000202)
Supervisor	Prof. Santosh Kumar, Associate Professor, CSE

Executive Summary

Farmers in Chhattisgarh who wish to purchase agricultural inputs online face a compounded barrier: conventional e-commerce interfaces are text-heavy, assume digital literacy, and operate in Hindi or English rather than Chhattisgarhi, an Eastern Hindi Indo-Aryan language spoken by more than sixteen million people that has its own vocabulary, pronouns and verb forms. This proposal describes a B.Tech Minor Project that builds and evaluates a voice-first Chhattisgarhi shopping assistant restricted to a single, well-defined workflow: product discovery, price and quantity queries, and conversational cart management for a curated agricultural product catalogue.

The proposed academic deliverable is deliberately narrow. It is not a payments-enabled marketplace, not a multi-vendor platform, and not an autonomous agronomic advisor. It is a working mobile prototype in which a farmer can speak in Chhattisgarhi, have the system correctly recognise the request, retrieve or calculate the relevant product information from a structured database, and hear a Chhattisgarhi voice response, while every diagnostic or dosage-related agricultural question is deliberately redirected to a human expert at the local Krishi Vigyan Kendra (KVK) rather than answered by the AI.

The team has already built and locally run a working Chhattisgarhi speech-to-speech pipeline — a FastAPI-based streaming speech-to-text service using Meta's MMS (Massively Multilingual Speech) model with a Chhattisgarhi (hne) adapter, and a Coqui-TTS VITS-based Chhattisgarhi speech synthesiser with trained male and female voice checkpoints. This preliminary work is used in this proposal as evidence that the core voice-interaction component is technically demonstrated, and it is deliberately reused as the primary technical foundation of the Minor Project rather than being replaced by unconfirmed third-party APIs.

Because the project currently has no allocated budget and no confirmed access to the Bhashini government API or to any paid large-language-model (LLM) service, the technology strategy prioritises free, open-source, and locally runnable components wherever an academically defensible open alternative exists, and explicitly marks any component that would depend on paid or unconfirmed infrastructure as a future, institutionally-approved possibility rather than a current deliverable.

The Minor Project's research contribution is framed around evaluating a cascaded, low-resource-language conversational pipeline for agricultural commerce — specifically, how accurately Chhattisgarhi speech can be converted into structured, safety-bounded shopping actions, and what accuracy, latency, and reliability trade-offs this cascaded architecture exhibits. The proposal also

outlines, separately and explicitly, a longer-term product vision in which the validated MVP could expand into a broader regional-language agricultural commerce platform; this future vision is presented as a possible follow-on direction and is not claimed as part of the current four-month academic deliverable.

1. Background and Motivation

Smartphone penetration in rural Chhattisgarh has increased substantially over the past decade, yet the interfaces that mediate access to agricultural commerce and information have not kept pace with the linguistic and literacy realities of their intended users. Government portals such as e-NAM and Kisan Suvidha, and private agri-marketplaces such as DeHaat, are built around Hindi- or English-language, text-and-menu navigation. For a smallholder farmer with limited formal education, typing agricultural terminology on a small touchscreen keyboard is slow and error-prone, and the underlying language of the interface is frequently not the farmer's first language.

Chhattisgarhi is grammatically and lexically distinct from standard Hindi. It uses its own pronouns (for example, “mola” for “to me” and “tor” for “yours”) and its own verb conjugation patterns, and a large share of its speakers use it in preference to Hindi in everyday conversation, particularly in rural and semi-urban areas. Voice assistants and speech interfaces built for standard Hindi routinely mis-transcribe Chhattisgarhi speech, and this friction pushes users away from otherwise-useful digital services.

Recent developments in Indic speech and language technology are changing what is feasible for a low-resource language such as Chhattisgarhi. Meta's Massively Multilingual Speech (MMS) project released acoustic models with adapters for over a thousand languages, including a Chhattisgarhi ('hne') adapter, which the team has already used locally. IISc's SPIRE Lab, in partnership with ARTPARK, has released large open Chhattisgarhi speech resources through the Project Vaani, RESPIN and SYSPIN initiatives, and — most recently, in August 2026 — an open-source multilingual ASR model named SraVaani-1.0 that explicitly includes low-resource Indian dialects. Meta's NLLB-200 translation model natively supports Chhattisgarhi under the FLORES-200 code hne_Deva. These developments mean that, for the first time, a student team can realistically assemble a Chhattisgarhi voice pipeline predominantly from free and open resources rather than depending on a single commercial vendor.

This Minor Project sits at the intersection of three concerns that the team considers individually well understood but rarely combined in a single, rigorously scoped student project: (a) low-resource speech and language processing for an under-served Indian dialect, (b) safety-constrained

conversational AI for a domain — agriculture — where incorrect automated advice carries real-world consequences, and (c) a transactional, stateful e-commerce backend. The proposal that follows is written to keep these three concerns in careful balance rather than allowing any one of them (particularly the AI) to be oversold.

2. Problem Statement

Digitally and linguistically marginalised farmers in Chhattisgarh are effectively excluded from convenient online purchase of agricultural inputs such as seed, fertiliser and basic tools, because the dominant e-commerce and agri-tech interfaces available to them (i) are optimised for text entry rather than speech, (ii) are built for Hindi or English rather than Chhattisgarhi, and (iii) provide no mechanism for a user to converse naturally about what they need, how much of it they need for a given field size, and whether it is available and at what price.

At the same time, any voice-based agricultural assistant carries a distinct hazard that a generic e-commerce voice assistant does not: if the assistant is allowed to behave as an open-ended conversational agent, it may be asked diagnostic or dosage questions (“which pesticide should I use for this disease?”) that a large language model can answer fluently but incorrectly, with consequences ranging from crop loss to health and environmental harm. A system that is voice-enabled but not safety-bounded is therefore not an acceptable design for this domain.

The problem this Minor Project addresses is consequently twofold and inseparable: (1) building a voice interface that lets a Chhattisgarhi-speaking farmer discover agricultural products and manage a shopping cart by speech alone, with acceptable accuracy and latency on a constrained student budget, and (2) constraining that same interface so that it functions strictly as a retail/discovery assistant and never as an autonomous source of agronomic diagnosis or dosage advice.

3. Rationale

The rationale for restricting the academic deliverable to voice-first product discovery and cart management, rather than attempting a full agricultural marketplace, follows directly from a feasibility review of what a three-member B.Tech team can responsibly deliver, test, and defend within a single semester-scale (approximately sixteen-week) window.

- A full multi-vendor marketplace with live inventory synchronisation, real payments, and logistics is a multi-year engineering effort even for a funded startup team, and attempting it inside a Minor Project would guarantee an unfinished, unevaluable deliverable.

- The team's own preliminary work already demonstrates that the hardest novel component — real-time Chhattisgarhi speech recognition and synthesis on local, open-source models — is achievable, which materially de-risks the voice layer of the project relative to the e-commerce layer.
- A narrow, well-instrumented scope (search, cart, price/quantity queries, simulated checkout) allows the team to collect meaningful, honestly-reported evaluation metrics (Word Error Rate, intent accuracy, task completion rate) instead of producing an impressive-looking but unmeasured system.
- Restricting the AI to a retail-assistant role, with hardcoded safety refusals for diagnostic agronomic queries, keeps the academic prototype within a defensible risk envelope and avoids the liability associated with automated pesticide or dosage recommendations.

Building the technically rigorous, safety-bounded MVP first, and treating commercialisation-oriented features (payments, vendor onboarding, logistics, broader regional-language support) as an explicit future-scope discussion rather than a current commitment, is the approach recommended by the team's own feasibility research and is retained as the guiding principle of this proposal.

4. Existing Systems and Competitive Analysis

Table 1 summarises the closest existing systems in the Indian agri-tech and e-commerce space, based on their publicly documented capabilities. No claim in this table asserts internal or proprietary knowledge of these platforms; it reflects publicly observable functionality only.

Platform	Purpose	Language Support	Voice Support	Agriculture Focus	Key Limitation for This Use Case
e-NAM	Government B2B commodity marketplace	Hindi, English, some regional	None	Yes (B2B trading)	No Chhattisgarhi; requires high digital literacy; not a farmer-facing retail voice UX
Kisan Suvidha	Government agricultural information portal	Hindi, English	None	Yes (informational)	Text-heavy; informational only, no transactional cart or purchase flow

Platform	Purpose	Language Support	Voice Support	Agriculture Focus	Key Limitation for This Use Case
DeHaat	Private agri-input marketplace	Hindi, English	Limited (Hindi)	Yes	No Chhattisgarhi support; voice is not the primary interaction mode
Amazon / Flipkart	General e-commerce	Hindi, English, others	Yes (Hindi)	Marginal	No agricultural product ontology; no Chhattisgarhi voice/NLP support
Kisan Call Centre	Human agronomic advisory helpline	Multilingual (human operators)	Yes (human)	Yes (advisory)	Relies on human operators; no automated e-commerce backend

The comparison clarifies what this project actually contributes: none of the above systems combine Chhattisgarhi-language voice interaction with a transactional agricultural shopping workflow. The gap is not “an agricultural marketplace” — several already exist — but the conversational, first-language modality of access to a workflow of that kind for a specific, currently under-served linguistic population.

5. Research Gap

Prior computational-linguistics work on Chhattisgarhi has largely treated speech recognition and translation as isolated benchmarking tasks — for instance, the RESPIN corpus and associated baseline ASR models evaluate word error rate on read speech, and NLLB-200 is evaluated on general-domain sentence translation. There is comparatively little published work that studies a multi-turn, state-aware, transactional agent that must correctly extract compound agricultural entities (crop variety, area, unit, quantity) from informal, code-switched Chhattisgarhi speech and preserve that state correctly across an e-commerce cart workflow.

The academic contribution of this Minor Project is therefore not the existence of an agricultural shopping application, nor the existence of Chhattisgarhi ASR or TTS in isolation — both already exist in various forms — but the integration and empirical evaluation of a cascaded, low-resource-language voice pipeline (ASR → translation → constrained LLM tool-calling → deterministic

backend → translation → TTS) as applied to a transactional agricultural-commerce state machine, together with an explicit, tested safety boundary that separates permitted shopping queries from disallowed agronomic-diagnostic queries.

6. Research Questions

The project is organised around the following research questions, each of which has a corresponding evaluation metric described in Section 24 (Evaluation Methodology):

1. RQ1: How accurately can spontaneous, informal Chhattisgarhi speech be converted into correctly structured shopping intents and entities (product, crop, quantity, unit) using a cascaded ASR → translation → LLM pipeline built primarily from open-source components?
2. RQ2: How does each stage of the cascade (ASR transcription error, machine-translation drift, LLM entity-extraction error) individually contribute to end-to-end intent-and-entity accuracy?
3. RQ3: Can a constrained, tool-calling LLM architecture reliably convert informal Chhattisgarhi farmer commands into valid, schema-conformant e-commerce operations (search, add-to-cart, remove-from-cart, view-cart) without hallucinating non-existent products or unsafe agronomic advice?
4. RQ4: What is the practical latency of a locally-hosted, zero-budget voice pipeline compared to an API-assisted alternative, and what latency is acceptable to a first-time voice-commerce user?
5. RQ5: Is a voice-first interface measurably more usable, for the target persona of a low-digital-literacy Chhattisgarhi-speaking farmer, than a conventional text-based product search interface for the same catalogue?

7. Objectives

The following objectives are deliberately limited to four, each stated in measurable and time-bound terms consistent with the sixteen-week academic schedule described in Section 22.

Objective 1 — Extend the existing Chhattisgarhi voice pipeline into a product-domain pipeline

Adapt the team's already-working MMS-based ASR and VITS-based TTS pipeline, plus a Chhattisgarhi–English translation stage, into a pipeline specialised for agricultural shopping vocabulary, and measure Word Error Rate (WER) and Character Error Rate (CER) on a project-specific evaluation set by Week 10.

Objective 2 — Build a constrained conversational shopping agent

Implement an LLM-based, strict tool-calling intent/entity extraction layer supporting at minimum the SEARCH_PRODUCT, PRODUCT_DETAILS, CALCULATE_REQUIRED_QUANTITY, ADD_TO_CART, REMOVE_FROM_CART, VIEW_CART and CHECK_PRICE intents, and measure intent-classification accuracy and entity-extraction F1 on a held-out evaluation set by Week 12.

Objective 3 — Deliver a working end-to-end mobile prototype

Deliver a Flutter mobile application connected to a FastAPI backend and a relational agricultural product database, capable of a complete spoken interaction from product query to simulated cart checkout, demonstrated live by Week 15.

Objective 4 — Enforce and evaluate an agricultural safety boundary

Design and test a safety-boundary mechanism that reliably distinguishes permitted shopping queries from disallowed diagnostic/dosage queries and redirects the latter to a Krishi Vigyan Kendra referral message, and report the boundary's detection accuracy on a labelled query set by Week 14.

8. Target Users

The system is designed around three user personas, adapted from the team's feasibility research, representing the range of digital-literacy and connectivity conditions expected among Chhattisgarhi-speaking farmers in and around Chhattisgarh, and the adjoining Chhattisgarhi-speaking regions of Madhya Pradesh and Odisha.

Persona 1: Smallholder farmer with limited digital literacy

Farms one to three acres, relies on rainfed agriculture, speaks primarily Chhattisgarhi, and finds standard grid-and-menu e-commerce apps difficult to navigate. This user depends on the voice assistant for essentially the entire shopping interaction and needs the interface to be forgiving of informal, hesitant, or incomplete spoken commands.

Persona 2: Medium-scale farmer with intermediate digital literacy

Manages upwards of five acres, frequently code-switches between Hindi and Chhattisgarhi, is comfortable with basic smartphone navigation, but prefers voice for quickly checking product availability or calculating input quantities for a given area rather than typing and scrolling.

Persona 3: Intermittent-connectivity user

Operates in a rural zone with fluctuating 3G/4G connectivity. This persona motivates the requirement, described in Section 19, that the mobile client degrade gracefully — with clear feedback and local caching — rather than fail silently when a voice request times out.

9. Proposed Solution

The proposed solution is a Chhattisgarhi voice-first agricultural product discovery and shopping-cart assistant, delivered as a lightweight mobile application backed by a REST API and a relational database. A farmer speaks a request in Chhattisgarhi; the system transcribes it, extracts the shopping intent and any product/quantity/area entities, executes the corresponding deterministic backend operation, and replies with a spoken Chhattisgarhi response. The core interaction loop is summarised below.

9.1 Conceptual interaction flow

- Farmer speaks a request in Chhattisgarhi (e.g. a search, a quantity question, or an add/remove-from-cart command).
- On-device voice activity detection (VAD) identifies the utterance boundary and streams compressed audio to the backend.
- A Chhattisgarhi automatic speech recognition (ASR) stage transcribes the audio to Devanagari text.
- A machine-translation stage normalises the Chhattisgarhi text into English (necessary because current LLMs reason far more reliably in English than in a low-resource language, as discussed in Section 13.3).
- A constrained, tool-calling LLM classifies the intent, extracts entities, and — if the query is a disallowed agronomic-diagnostic question — triggers a safety refusal instead of a database operation.
- For a permitted shopping intent, the backend performs the corresponding deterministic operation (search, quantity calculation from verified product metadata, cart update) against the PostgreSQL product/cart database.
- The result is translated back into Chhattisgarhi and rendered as speech through the TTS stage, and streamed back to the mobile client.

Figure 1: *Conceptual voice interaction pipeline — Speech → ASR → MT (Chhattisgarhi→English) → LLM intent/entity extraction → backend tool call → MT (English→Chhattisgarhi) → TTS → Speech response.*

This cascaded architecture is chosen over a hypothetical end-to-end speech-to-speech model because no open-source model currently combines native Chhattisgarhi audio understanding, agricultural-domain reasoning, and Chhattisgarhi speech synthesis in a single system with sufficient reliability; the cascaded design lets each stage be built from an existing, independently-verifiable open component (Section 13 discusses this choice, and the alternatives considered, in more detail).

10. System Scope: Three Levels of Maturity

To avoid any ambiguity about what is being promised in this Minor Project, the proposal explicitly separates the work into three levels. This separation is maintained consistently through every later section of the document.

Level 1 — Already Demonstrated (Preliminary Work)

A working, locally-run Chhattisgarhi speech-to-speech pipeline already exists, built by the team prior to this proposal. It performs real-time streaming ASR using Meta's MMS model with a Chhattisgarhi adapter, and speech synthesis using a Coqui-TTS VITS model with trained male and female Chhattisgarhi voice checkpoints. Full technical detail is given in Section 30 (Preliminary Work). This is evidence of feasibility for the voice-interaction component only — it does not include e-commerce, agricultural entities, or safety-boundary logic, all of which are new work for this Minor Project.

Level 2 — Minor Project MVP (This Proposal's Deliverable)

The system to be designed, built and evaluated between now and approximately mid-December 2026: Chhattisgarhi voice input for product search, product details, deterministic quantity calculation, conversational add-to-cart and remove-from-cart, cart viewing, price checking, and a simulated (non-payment) checkout, delivered through a basic Flutter mobile application and a FastAPI backend, with the safety boundary described in Section 18 enforced throughout.

Level 3 — Future Startup / Product Direction (Explicitly Out of Scope Now)

A possible longer-term evolution of the validated MVP into a broader regional-language agricultural commerce platform, potentially including real UPI/payment integration, multi-vendor inventory, logistics and delivery tracking, farmer profiles, additional regional languages, and safety-controlled personalised recommendations. Level 3 is discussed only in Section 29 (Startup and Future Expansion) as a future possibility contingent on institutional support and is not claimed as part of the current academic deliverable.

11. MVP Feature Set (Functional Requirements)

Table 2 lists the functional requirements of the Level 2 MVP. Every row is scoped to what the team can realistically implement and evaluate within the sixteen-week timeline; features that were considered but excluded from the MVP are listed afterwards for transparency.

Requirement ID	Description	Priority
FR-1	Accept streamed Chhattisgarhi speech input from the mobile client with on-device VAD	Must-have
FR-2	Transcribe Chhattisgarhi speech to Devanagari text (ASR)	Must-have
FR-3	Translate Chhattisgarhi text to English and back (MT)	Must-have
FR-4	Classify shopping intent and extract entities via constrained LLM tool-calling	Must-have
FR-5	Voice-based product search and retrieval from the catalogue	Must-have
FR-6	Voice-based product detail / price lookup	Must-have
FR-7	Deterministic area-to-quantity calculation using verified product metadata (e.g. seed rate per acre)	Must-have
FR-8	Conversational add-to-cart and remove-from-cart	Must-have
FR-9	Conversational cart viewing (contents, quantities, running total)	Must-have
FR-10	Simulated checkout (order recorded in database; no real payment)	Must-have
FR-11	Synthesise and stream a Chhattisgarhi voice response (TTS)	Must-have
FR-12	Safety boundary: detect and refuse diagnostic/dosage agronomic queries, redirecting to KVK	Must-have
FR-13	Basic user authentication (JWT-based)	Must-have
FR-14	Logging of voice/intent/latency data for evaluation (VoiceLogs)	Must-have
FR-15	Basic mobile UI showing recognised text, cart state, and product images for confirmation	Should-have
FR-16	Graceful handling of network timeouts with a pre-recorded Chhattisgarhi fallback prompt	Should-have
FR-17	Multi-turn contextual memory (e.g. recalling a crop mentioned earlier in the conversation)	Could-have (Version 2, time permitting)

Requirement ID	Description	Priority
FR-18	Voice-based order-status query	Could-have (Version 2, time permitting)

Explicitly excluded from the current MVP

Consistent with the team's feasibility analysis and with the zero-budget, three-student, sixteen-week constraint, the following are not part of the current academic deliverable and are discussed only under Future Scope (Section 29):

- Real payment gateway / UPI integration
- Live multi-vendor inventory synchronisation
- Seller/vendor dashboards
- Delivery logistics, routing, or OTP-based delivery confirmation
- Open-ended agronomic advisory AI (diagnosis, dosage recommendation)

12. Non-Functional Requirements

Category	Requirement	Target / Note
Latency	End-to-end voice round-trip time (speech in → speech response out)	Target < 5 s; degrade gracefully above this, per Section 19
Reliability	Client-side request timeout with fallback audio prompt	8 s timeout, matches the team's own feasibility recommendation
Security	API keys and credentials never shipped in client code; JWT-based auth on all endpoints	Enforced from Phase 2 onward
Privacy	No PII (names, phone numbers, precise addresses) captured in evaluation voice recordings	Enforced under the data collection protocol, Section 20
Data integrity	Cart and inventory operations must be transactional (ACID) to prevent overselling or corrupted state	PostgreSQL with explicit transactions

Category	Requirement	Target / Note
Portability	Mobile client runs on commodity Android devices representative of rural users	Flutter cross-platform build
Cost	No component may require paid infrastructure without being explicitly flagged 'subject to institutional approval'	See Section 28, Budget
Auditability	Every voice interaction is logged (transcript, intent, latency, outcome) for evaluation	VoiceLogs table, Section 14.2

13. Proposed System Architecture

The architecture follows a decoupled, layered design: a Flutter mobile client for audio capture and UI, a FastAPI backend that orchestrates the voice pipeline and exposes the e-commerce API, and a PostgreSQL database for products, users, carts and orders. This section discusses the voice pipeline options considered and the recommendation for this Minor Project.

13.1 Voice pipeline: architecture options considered

Option	Description	Assessment for this project
Option A — Cascaded pipeline (recommended)	ASR → MT → LLM tool-calling → backend → MT → TTS, using independently-verifiable open components at each stage	Feasible on student hardware and zero budget; every stage already has a working or freely available open-source implementation; chosen for this MVP
Option B — End-to-end multimodal LLM	A single model natively ingesting Chhattisgarhi audio and producing Chhattisgarhi audio, reasoning over agricultural context throughout	No open-source model currently combines native Chhattisgarhi audio understanding and synthesis with sufficient reliability; not feasible within this timeline and is noted only as a long-term research direction

13.2 Recommended technology stack: local/open-source vs. API-assisted

Because the team currently has no allocated budget and no confirmed access to the Bhashini government API or to a paid LLM service, every layer of the stack is evaluated as a local/open-source option versus an API-assisted option, and a practical hybrid strategy is recommended rather than assuming paid infrastructure will be available.

Layer	Local / Open-Source Option (default for MVP)	API-Assisted Option (contingent on future access)
ASR	Meta MMS (facebook/mms-1b-all) with the Chhattisgarhi 'hne' adapter — already integrated and running locally by the team; SraVaani-1.0 (ARTPARK/IISc, open-source, released August 2026, 65 Indian languages/dialects, ~430M parameters) is a promising newer alternative earmarked for benchmarking in Phase 6	Bhashini ASR API (subject to registration and confirmed academic access — see Section 31)
Machine Translation	Meta NLLB-200 (distilled 600M checkpoint), which natively supports Chhattisgarhi under the FLORES-200 code hne_Deva, run locally via CTranslate2/Transformers	Bhashini MT API (subject to confirmed access); IndicTrans2 is not used as the primary MT model because it targets India's 22 Eighth-Schedule scheduled languages and does not officially include Chhattisgarhi — any use of IndicTrans2 for Chhattisgarhi would rely on unverified zero-shot cross-lingual transfer
TTS	Coqui-TTS VITS model with the team's own trained Chhattisgarhi male/female checkpoints — already working locally	Bhashini TTS API / SYSPIN-derived models (subject to confirmed access)
Intent & entity extraction (LLM)	A locally-hosted open-weight LLM with function/tool-calling support, run via a lightweight local inference server on available student hardware, used for the initial prototype	A hosted LLM API selected at implementation time based on available free-tier or academic credits, if and when confirmed — no specific paid vendor is committed to in this proposal
Backend	Python (FastAPI)	— (no change; open-source in both scenarios)
Database	PostgreSQL, self-hosted or via Supabase free tier	— (open-source / free-tier in both scenarios)
Mobile client	Flutter	— (open-source in both scenarios)

Layer	Local / Open-Source Option (default for MVP)	API-Assisted Option (contingent on future access)
Hosting	Free/hobby tiers of a platform such as Render or Railway, or institute lab hardware where available	Paid tier only if institutionally sponsored — marked ‘subject to approval’

This table intentionally does not force the Bhashini ecosystem or a specific paid LLM vendor into the recommended path, because neither is currently confirmed. The default MVP path is built entirely on components the team can run today without external approval: MMS, NLLB-200, Coqui-TTS, and a locally-hosted open-weight LLM. If Bhashini access and/or LLM API credits are confirmed during Phase 1 (Weeks 1–2), the team will benchmark the API-assisted alternative against the local baseline and report both, since comparing the two is itself one of the project's research questions (RQ4).

13.3 Why a translation step is used

Contemporary LLMs exhibit markedly weaker reasoning, tool-calling reliability, and context retention when prompted directly in a low-resource language such as Chhattisgarhi, compared with English, because their pretraining data is overwhelmingly English-dominant. Routing the transcribed Chhattisgarhi text through a Chhattisgarhi–English translation stage before LLM reasoning is therefore treated as a necessary architectural choice rather than an optional one, and the translation stage's own error contribution is measured separately as part of RQ2.

13.4 Complete architecture flow

The mobile client records audio, applies local VAD, and streams compressed (Opus-codec) audio to the FastAPI gateway. The gateway dispatches the audio to the ASR stage, passes the resulting Chhattisgarhi text through the MT stage, and sends the English text — together with a system prompt defining the permitted tool schema and the safety boundary — to the LLM. When the LLM emits a tool call (for example `add_to_cart`), the backend executes the corresponding SQL transaction against PostgreSQL, returns a structured result to the LLM for response generation, translates the English response back to Chhattisgarhi, synthesises audio via TTS, and streams the response back to the client.

Figure 2: *Backend orchestration: Mobile client ↔ FastAPI Gateway ↔ {ASR, MT, LLM tool-calling, TTS} ↔ PostgreSQL (Products, Cart, Orders, VoiceLogs).*

14. Agricultural Product Data Model and Database Architecture

14.1 Rationale for a deterministic, rule-based recommendation layer

A central design principle of this project, carried directly from the team's feasibility research, is that agricultural quantity recommendations must never be produced from an LLM's internal knowledge. The LLM's role is strictly limited to natural-language routing (understanding what the farmer is asking for); any quantity calculation is performed by ordinary, auditable Python/SQL logic operating on verified product metadata (for example, a seed-rate-per-acre field on the product record). This separation is what allows the system to answer “how much seed do I need for two acres?” safely while refusing to answer “what should I spray on my diseased crop?” — the former is arithmetic over a verified static field, the latter would require the LLM to diagnose a disease it cannot see and has not been trained to safely diagnose.

Note on data provenance: illustrative examples in this proposal reference Indira Gandhi Krishi Vishwavidyalaya (IGKV) as the intended authoritative source for regional crop and seed-rate metadata. Specific variety names, seed-rate figures, and agronomic parameters have not yet been independently sourced or verified by the team at proposal stage and must be obtained directly from IGKV publications or extension materials during Phase 1 before being encoded into the product database; no such figures are asserted as verified fact in this document.

14.2 Proposed relational schema

Table	Primary Key	Key Attributes
Users	user_id	phone_number, name, language_pref, role
Products	product_id	name, description, price, stock, unit, crop_compatibility, dosage_per_acre / seed_rate_per_acre
Categories	category_id	category_name (e.g. Seed, Fertiliser, Tool)
Cart	cart_id	user_id (FK), status, created_at, updated_at
CartItems	cart_item_id	cart_id (FK), product_id (FK), quantity, price_at_addition
Orders	order_id	user_id (FK), total_amount, status, created_at
OrderItems	order_item_id	order_id (FK), product_id (FK), quantity, price_locked
VoiceLogs	log_id	audio_ref, transcribed_text, translated_text, intent_detected, entities_json, latency_ms, outcome

Figure 3: Simplified entity-relationship overview: *Users → Cart → CartItems → Products → Categories; Users → Orders → OrderItems; VoiceLogs referencing Users and, where applicable, the resulting cart/order action, for evaluation traceability.*

PostgreSQL is selected for its relational integrity guarantees (ACID transactions), which are important for correctly handling concurrent cart updates and preventing stock/quantity inconsistencies, and because a managed free tier (e.g. via Supabase) removes the need for the team to operate its own database server.

15. Backend and API Design

The backend is implemented in Python using FastAPI, chosen for its native asynchronous request handling, which matters because ASR/MT/LLM/TTS calls in the voice pipeline are all comparatively slow, blocking operations relative to typical CRUD calls. The backend exposes a small, well-defined REST/WebSocket surface: authentication, product search/detail endpoints, cart endpoints, an order-simulation endpoint, and a streaming voice endpoint that internally sequences the ASR → MT → LLM → MT → TTS cascade described in Section 13.4. Server-Sent Events (or WebSockets) are used to stream intermediate pipeline progress to the client so that the user receives feedback (e.g. a brief audio cue) while the multi-second pipeline is still processing, rather than experiencing a silent, unexplained delay.

Prompt-injection resistance is addressed structurally rather than through instruction alone: the LLM is constrained to emit only predefined JSON tool-call schemas or short conversational text, so a user attempting to instruct the model to “ignore previous instructions” cannot cause the backend to execute an operation outside the fixed set of permitted tools.

16. Mobile Application

The mobile client is built in Flutter for cross-platform reach with a single codebase, and because Flutter's audio-recording ecosystem (e.g. the record package) is mature enough for a three-person team to integrate within the available time, compared with the steeper native-audio integration effort that React Native or platform-native development would require. The client's responsibilities are: capturing and locally VAD-gating microphone audio, compressing it (Opus codec) before upload to minimise bandwidth use on constrained rural networks, rendering a minimal visual confirmation of what was understood (recognised text, product images, running cart total) so the farmer is not solely dependent on audio memory, and caching recently viewed product metadata and low-resolution images locally so the app remains partially usable while a voice request is in flight or the network is degraded.

17. AI/ML Development Strategy

The academic timeline rules out training a speech or language foundation model from scratch; the AI/ML strategy is therefore built entirely around adapting and evaluating existing open models rather than training new ones.

- ASR and TTS: reuse and extend the team's already-trained MMS/VITS Chhattisgarhi models (Level 1 preliminary work), fine-tuning or prompt/vocabulary-biasing them toward agricultural and shopping vocabulary using the RESPIN and SYSPIN corpora described in Section 20.
- Translation: use NLLB-200 directly, since it has verified native Chhattisgarhi support (hne_Deva); no fine-tuning is planned for the MVP, but translation-quality evaluation against the project's own evaluation set is planned (RQ2).
- Intent/entity extraction: use prompt engineering and strict JSON-schema tool-calling over an existing instruction-tuned LLM rather than training a traditional intent classifier (e.g. Rasa/spaCy-style), because traditional classifiers require tens of thousands of annotated examples that do not exist for Chhattisgarhi agricultural commerce, whereas a general-purpose LLM can be steered with a comparatively small number of few-shot examples once the Chhattisgarhi query has been translated into English.
- Recommendation logic: rule-based only, implemented in Python/SQL against verified product metadata, with no LLM involvement in any numeric agronomic calculation, consistent with Section 14.1.
- Retrieval-augmented generation (RAG) is deliberately not used: the product catalogue is a small, structured relational dataset (well under a thousand items), for which direct SQL/full-text search is simpler, faster, and easier to audit than a RAG pipeline, and RAG's added architectural complexity is not justified at this scale.

18. Safety, Privacy and Security

18.1 Agricultural safety boundary (critical design requirement)

This system is designed, described, and evaluated strictly as a shopping and product-discovery assistant. It is explicitly not designed, and must never be presented, as an autonomous agricultural doctor or agronomic advisor. This distinction is treated as a first-class requirement of the architecture rather than an afterthought, because an AI system that hallucinates a pesticide type or dosage in response to a diagnostic question could cause real crop loss, financial harm, or health and environmental risk.

The system prompt governing the LLM explicitly separates permitted and disallowed query categories:

Permitted (retail / discovery) queries

- “Show me urea fertiliser.” — catalogue search
- “What seeds are available?” — catalogue browse
- “Add two bags of fertiliser.” — cart operation
- “How much seed is required for two acres?” — permitted only because it is answered by deterministic arithmetic over verified, static IGKV-sourced metadata, never by LLM inference

Disallowed (diagnostic / dosage) queries — trigger a safety refusal

- “Which pesticide should I use for this disease?”
- “What chemical will cure this crop?”
- “How much pesticide should I apply to my diseased plants?”

When a disallowed query is detected, the LLM is constrained to emit a fixed, hardcoded referral response directing the farmer to the local Krishi Vigyan Kendra (KVK) rather than allowing free-form generation, and the backend logs the refusal for later evaluation of the boundary's detection accuracy (Objective 4, Section 7). This behaviour is treated as a core deliverable, evaluated with the same rigor as the shopping functionality, and is discussed again under Risks (Section 26) and Future Scope (Section 29), where any expansion toward personalised agronomic assistance is explicitly deferred and marked as requiring qualified domain-expert and institutional oversight before being considered.

18.2 Security

- Authentication and authorisation via JWT tokens issued by the backend on login; all cart/order endpoints require a valid token.

- API keys and any third-party credentials (translation/ASR/TTS or LLM API keys, if and when confirmed) are stored only in backend environment variables and are never bundled into the mobile client.
- The LLM is structurally restricted to emitting predefined JSON tool-call schemas or short conversational text, which limits the impact of prompt-injection attempts, since the model cannot be made to perform an operation outside the fixed tool set regardless of what a user asks it to 'ignore'.
- Standard encryption-at-rest for the database is used via the chosen hosting provider's default configuration (e.g. Supabase-managed PostgreSQL).

18.3 Privacy

- Order and cart data in the MVP is simulated (no real payment or fulfilment), and no financial instrument data is collected.
- Voice recordings collected for evaluation purposes (Section 20) exclude personally identifiable information by protocol — real names, addresses and financial details are not to be spoken or recorded during data collection.
- Informed consent is obtained from every participant prior to any evaluation recording, consistent with standard academic research ethics practice for human-subjects speech data collection.

19. Offline and Low-Connectivity Strategy

Fully offline Chhattisgarhi ASR on a low-end rural smartphone is not realistic within this project's scope: even a comparatively compact ASR model requires more RAM and compute than is reliably available on typical low-end Android hardware, and running the LLM reasoning step on-device is not feasible for a lightweight Flutter application. The system therefore assumes a connected architecture and instead focuses on graceful degradation rather than true offline operation.

- Audio is compressed with the Opus codec before upload to minimise bandwidth demand on constrained 3G/4G rural networks.
- Recently viewed product text and low-resolution images are cached locally on the client so the user can browse previously seen catalogue information even while a new voice request is in flight or the network is briefly unavailable.
- The client enforces an 8-second request timeout; on timeout, the app aborts the pending request and plays a locally stored, pre-recorded Chhattisgarhi audio prompt explaining that the network is unavailable and inviting the user to try again, rather than freezing or failing silently.

Overpromising full offline conversational AI would misrepresent the project's actual feasibility; this proposal deliberately states the offline limitation plainly rather than implying a capability that cannot be delivered.

20. Dataset and Data Collection Strategy

20.1 Existing open datasets to be used (verified, publicly available)

A web-based verification check was carried out before finalising this strategy. The following resources were confirmed to be real, publicly documented, and freely downloadable at proposal time, and are used in preference to any dataset the team would otherwise need to collect from scratch:

- RESPIN-S1.0 (IISc SPIRE Lab, supported by the Gates Foundation): a public, dialect-rich read-speech corpus exceeding 10,000 validated hours across nine Indian languages including Chhattisgarhi, explicitly covering the agriculture and finance domains, with published baseline ASR benchmarks. This is a strong, directly relevant resource for evaluating and potentially fine-tuning the project's ASR stage on agriculture-domain Chhattisgarhi speech.
- SYSPIN_S1.0 (IISc SPIRE Lab, supported by GIZ, Germany): a public, studio-recorded Chhattisgarhi TTS corpus (part of a nine-language release exceeding 40 hours per language, one male and one female voice artist per language), usable as a reference or fine-tuning resource for the team's own VITS-based TTS models.
- NLLB-Seed / FLORES-200 evaluation data: small, human-translated evaluation sets that include Chhattisgarhi (hne_Deva), usable for sanity-checking translation quality.

Correction to the team's original research report: SraVaani-1.0 (ARTPARK/IISc) is confirmed real and open-source, but it was released in August 2026 — after the team's original feasibility research and close to this proposal's submission — and is a ~430M-parameter model, larger than the components already integrated in the team's Level 1 prototype. It is treated in this proposal as a promising benchmarking candidate for later phases rather than as a component the MVP currently depends on. Its reported outperformance of specific commercial models on certain language/dataset combinations comes from ARTPARK's own published evaluation and is cited as such, not independently re-verified by the team.

20.2 Project-specific data to be built by the team

- Agricultural product catalogue: a manually curated set of approximately 50–100 realistic Chhattisgarh-relevant agricultural products (seed, fertiliser, basic tools) with verified

metadata, sourced from IGKV publications and other authoritative regional agricultural references during Phase 1.

- Shopping-command evaluation set: a small, purpose-built set of Chhattisgarhi agricultural shopping commands (e.g. “add two bags of urea”) recorded from a modest number of native Chhattisgarhi speakers, used specifically to evaluate the end-to-end pipeline on the project's own shopping vocabulary, since RESPIN/SYSPIN cover general agriculture/finance domain speech but not this project's specific e-commerce phrasing.

20.3 Data collection ethics

- Informed consent obtained from every participant before recording.
- No personally identifiable information (real names, addresses, financial details) recorded as part of shopping-command samples.
- Recordings and derived transcripts stored securely and used only for project evaluation, not shared or published beyond the academic deliverable without further consent.

The team does not claim that any planned evaluation sample size constitutes a statistically significant or generalisable result; any sample size stated in the Implementation Timeline below is a planned data-collection target, and results obtained from it will be reported as indicative of a student research prototype, not as a validated production benchmark.

21. Methodology

The project follows a phased, build-and-evaluate methodology rather than a single upfront design-then-implement waterfall, because the team's confidence in several components (LLM tool-calling reliability, end-to-end latency, translation quality) can only be established empirically. Each phase in Section 22 produces a concrete, testable artefact, and the evaluation metrics defined in Section 24 are collected incrementally rather than only at the end of the project, so that a component found to be underperforming (for example, unacceptably high WER on a particular ASR configuration) can be revisited before it blocks integration.

Objective-wise, the methodology maps as follows: Objective 1 (voice pipeline) is addressed through Phases 1 and 3; Objective 2 (conversational agent) through Phase 3; Objective 3 (end-to-end mobile prototype) through Phases 2, 4 and 5; and Objective 4 (safety boundary) through Phase 3 and the dedicated safety-boundary evaluation in Phase 6.

22. Implementation Timeline

The project runs from the current point in the Autumn 2026 academic session to approximately mid-December 2026, a window of approximately sixteen weeks. Table 3 presents the phase-wise plan; Figure 4 presents the same plan as a Gantt-style overview.

Phase	Weeks	Key Tasks	Deliverable
Phase 1: Scoping & Research	1–2	Finalise product catalogue scope; source IGKV crop/seed-rate metadata; investigate Bhashini registration and any available LLM API/academic credits; design DB schema	Confirmed DB schema; product metadata sourcing plan; API-access status report
Phase 2: Backend & Database	3–4	Stand up PostgreSQL (Supabase or self-hosted); implement FastAPI CRUD for products/cart/orders; implement JWT auth	Working REST API for catalogue, cart and orders
Phase 3: Voice & Conversational AI Pipeline	5–8	Extend MMS/VITS pipeline with agricultural vocabulary; integrate NLLB-200 translation; implement LLM tool-calling for the intent taxonomy; implement and test the safety boundary	End-to-end pipeline processing Chhattisgarhi text/audio into valid tool calls, including safety-refusal behaviour
Phase 4: Mobile Application	9–11	Build Flutter UI; implement audio capture, VAD and Opus compression; implement cart/product UI and offline caching	Installable APK with working microphone input and cart UI
Phase 5: Integration	12–13	Connect mobile client to backend; implement SSE/WebSocket progress streaming; handle end-to-end state management and timeouts	Fully integrated, working end-to-end prototype
Phase 6: Evaluation	14–15	Collect evaluation voice sample set; measure WER/CER, intent accuracy, entity F1, task completion rate, latency, safety-boundary accuracy; conduct a small usability walkthrough if feasible	Evaluation report with quantitative metrics (Section 24)
Phase 7: Documentation & Demonstration	16	Compile final report, architecture diagrams and figures; prepare live demonstration	Final Minor Project report and demonstration

Figure 4: Gantt-style overview — Phases 1–7 run sequentially with short overlaps between adjacent phases (e.g. mobile UI work in Phase 4 begins once the core API contract from Phase 2 is stable); all three students converge on Phase 6 (Evaluation) and Phase 7 (Documentation).

23. Team Roles and Responsibilities

Responsibilities are allocated by primary technical strength, consistent with the division proposed in the team's own feasibility research, while all three members jointly participate in integration, testing, evaluation, documentation and the final demonstration.

Member	Primary Responsibilities
Punyansh Thakur (241020461)	AI/ML and speech pipeline lead: MMS/VITS pipeline extension, NLLB-200 integration, LLM prompt engineering and tool-calling schema design, safety-boundary logic, evaluation experiment design (WER/CER, intent/entity metrics)
Harsh Dadsena (241020433)	Backend and database lead: FastAPI architecture, PostgreSQL schema and transactions, deterministic quantity-calculation logic, authentication and API security, deployment
Aakash Sen (241000202)	Frontend/mobile and integration lead: Flutter application, audio capture/VAD/compression, UI/UX for voice and cart interactions, offline caching, client–backend integration

All three students collaborate during Phase 6 on evaluation-data collection and metric computation, and during Phase 7 on the final report and demonstration. This allocation reflects the team's current preference and may be adjusted as the project proceeds; any material change will be reflected in project documentation and communicated to the supervisor.

24. Evaluation Methodology

Table 4 lists the metrics used to answer the research questions in Section 6. All figures reported at the end of the project will be actual measured results from the team's own evaluation set(s); no numeric result is asserted in this proposal document, since the system does not yet exist to measure.

Category	Metric	Purpose / Maps to
Speech (ASR)	Word Error Rate (WER), Character Error Rate (CER)	RQ1, RQ2 — measured on the project's Chhattisgarhi shopping-command evaluation set and, where applicable, on relevant RESPIN agriculture-domain subsets

Category	Metric	Purpose / Maps to
NLP / conversational understanding	Intent classification accuracy; entity-extraction precision/recall/F1; tool-call schema validity rate	RQ1, RQ3, Objective 2
Safety	Safety-boundary detection accuracy on a labelled set of permitted vs. disallowed queries	Objective 4
E-commerce	Product search success rate; add-to-cart / remove-from-cart success rate; quantity-calculation accuracy against verified metadata	Objective 3
System performance	End-to-end latency (time to first audio byte of the TTS response); API/model response time per stage; request failure and recovery rate	RQ4
Usability (where feasible)	Task completion rate; a brief structured feedback questionnaire with a small number of Chhattisgarhi-speaking participants	RQ5 — reported as a small, indicative pilot, not a statistically powered study

Where a planned sample size (for the evaluation voice set or a usability walkthrough) is mentioned in this document, it denotes a target for data collection, not an achieved result, and any conclusions drawn from a small sample will be reported with that limitation stated explicitly in the final report.

25. Expected Outcomes

- A working, demonstrable mobile prototype implementing the full MVP feature set described in Section 11, running end-to-end on the team's chosen local/open-source stack.
- A measured, honestly reported evaluation of the cascaded voice pipeline's accuracy and latency (Section 24), including a breakdown of error contribution by pipeline stage, directly answering RQ1–RQ2.
- A tested safety-boundary mechanism with measured detection accuracy, demonstrating that the assistant reliably distinguishes shopping queries from disallowed agronomic-diagnostic queries.
- A documented comparison between the zero-budget local/open-source configuration and an API-assisted configuration, if API access is confirmed during the project (RQ4), providing evidence for future architecture decisions.

- A final project report and live demonstration suitable for academic evaluation, together with a clearly separated discussion of the Level 3 future/startup direction (Section 29) that does not overstate what the Minor Project itself has delivered.

26. Risks and Mitigation (Failure-Mode Analysis)

Failure Mode	Cause	Impact	Detection & Mitigation
ASR mis-recognises Chhattisgarhi as Hindi or fails on code-switched speech	Acoustic model bias toward higher-resource languages	Entity extraction fails; local crop/product names lost	Fuzzy phonetic backend search attempted before failing; benchmark against SraVaani as an alternative in Phase 6
LLM invents a non-existent product (hallucination)	Unconstrained free-form generation	Cart corrupted with an invalid item	LLM restricted to strict JSON tool calls; backend performs the actual search and returns verified facts for the LLM to summarise; foreign-key constraints reject invalid product references
Quantity calculation is wrong	LLM attempts arithmetic internally instead of delegating to the backend	Over-ordering, or an unsafe/incorrect input-quantity recommendation	LLM is prohibited from performing agronomic arithmetic; all calculations run in backend Python/SQL against verified metadata; backend validates results against a maximum-plausible-quantity threshold
Network disconnects mid-request	Rural connectivity variability	App appears frozen, user frustration	8-second client timeout; abort and play a local pre-recorded Chhattisgarhi 'network error' prompt (Section 19)
Ambiguous spoken instruction (e.g. 'add it' with no referent)	Farmer omits required detail	System lacks context to execute the command	LLM prompted to ask a clarifying question rather than guessing

Failure Mode	Cause	Impact	Detection & Mitigation
End-to-end latency exceeds usability threshold	Cumulative delay across ASR/MT/LLM/MT/TTS stages	User perceives the system as broken during a multi-second pause	SSE/WebSocket progress streaming with an audible holding cue; LLM prompted for concise responses to reduce TTS generation time; latency measured and reported as a documented limitation if unresolved
Confirmed API access to Bhashini/LLM providers is delayed or denied	Institutional approval process outside the team's control	API-assisted comparison (RQ4) cannot be completed as planned	MVP is designed to be fully functional on the local/open-source path regardless of API outcome; the API-assisted comparison is treated as a stretch goal, not a dependency for core deliverables
Safety boundary under- or over-triggers	Ambiguous phrasing between a permitted quantity question and a disallowed diagnostic question	Either an unsafe response slips through, or a legitimate shopping query is wrongly refused	Boundary behaviour is treated as a first-class, separately evaluated metric (Section 24); refusal wording is fixed/hardcoded rather than freely generated to avoid unsafe drift

27. Budget

The project currently has no allocated budget (₹0). The technology choices throughout this proposal are made specifically to keep the MVP fully functional at zero direct cost to the team, using student hardware, free-tier cloud services, and open-source models. Table 6 itemises the anticipated cost posture; any item that would require paid infrastructure is explicitly marked as contingent on future institutional approval rather than assumed.

Component	Zero-Budget Approach (current MVP)	Potential Future Cost (subject to institutional approval)
Compute for ASR/TTS/LLM inference	Existing student hardware; speech-lab access where available on the institute's premises	Cloud GPU hosting, if lower latency or larger models are required at a later stage

Component	Zero-Budget Approach (current MVP)	Potential Future Cost (subject to institutional approval)
Cloud hosting (backend)	Free/hobby tier of a platform such as Render or Railway	Paid hosting tier for higher reliability, if sponsored
Database	Supabase free tier (PostgreSQL)	Paid managed-database tier at higher scale
Speech & translation APIs (Bhashini)	Not used in the default MVP path; registration is free and self-service, but the team has not yet completed registration or confirmed usage terms	Continued/expanded use, if registration and any usage terms are confirmed and approved
LLM API (if used instead of a local model)	Not committed to in the default MVP path; a locally hosted open-weight model is the default	A hosted LLM API, selected based on available free-tier or academic credit at implementation time — no vendor or price is assumed in this proposal
Domain name / misc.	Not required for an academic demonstration build	Nominal annual cost if a public-facing demo deployment is desired

Any figure for API pricing that the team encounters during Phase 1 investigation will be recorded against current, dated vendor documentation at that time and reported transparently in the project's interim documentation; no current API pricing is asserted in this proposal, since the team does not yet have confirmed account-level pricing terms.

28. Startup and Future Expansion (Level 3 — Not Part of the Current Deliverable)

The team recognises that a validated Chhattisgarhi voice-first agricultural shopping assistant addresses a real and currently under-served need, and that the underlying technology and user-experience pattern could, in principle, extend beyond the scope of this Minor Project. This section describes that possible future direction explicitly and separately, so that it is never confused with what is being delivered between now and mid-December 2026.

29.1 Current MVP (reiterated)

A Chhattisgarhi voice-based agricultural product-discovery and cart-management prototype, for a single curated catalogue, with simulated checkout and no live vendor or payment integration.

28.2 Possible future product direction

If the MVP successfully validates the core technology and user experience, a future, post-Minor-Project iteration could explore extending the platform toward a broader regional-language agricultural commerce product, potentially including:

- Real UPI/payment gateway integration
- Real-time, multi-vendor inventory synchronisation
- Additional agricultural input suppliers and categories
- Order tracking and delivery/logistics integration
- Support for additional regional Indian languages beyond Chhattisgarhi
- Personalised, but safety-controlled, agricultural assistance developed in structured collaboration with qualified agricultural experts and institutions such as IGKV or local KVKs, never as an unsupervised extension of the current LLM's role
- Farmer profiles and aggregate, privacy-respecting demand analytics

No market traction, revenue, customer commitments, partnerships, or business validation exist at the time of this proposal, and none are claimed. The intended sequencing is deliberate: build a technically rigorous, safety-bounded MVP first; validate the core user problem and the underlying technology through the evaluation described in Section 24; and only then consider whether a scaled, commercially-oriented product is warranted and, if so, pursue it as a separate effort with appropriate institutional, legal and business support beyond the scope of a B.Tech Minor Project.

29. Preliminary Work (Level 1 — Already Demonstrated)

Prior to this proposal, the team built and locally ran a working Chhattisgarhi speech-to-speech pipeline. This section documents its technical specifics as evidence of feasibility for the voice-interaction component of the Minor Project. No accuracy figures, benchmark numbers, or user-testing results are claimed beyond what is stated below; where a formal metric has not yet been calibrated, this is stated explicitly rather than estimated.

29.1 Speech-to-Text (STT) module

- Built with FastAPI to handle live, concurrent, low-latency WebSocket connections streaming binary audio (16kHz PCM).
- Uses Silero VAD for real-time voice activity detection, governing partial vs. final transcript emission and detecting user barge-in.
- Implements a pluggable ASREngine interface with a shared inference thread pool, allowing multiple callers to be served without reloading the model.
- Includes a telephony_adapter capable of intercepting 8kHz mu-law audio (e.g. from telephony networks such as Twilio), which is not required for the Minor Project's mobile-app scope but demonstrates the pipeline's underlying flexibility.

29.2 Text-to-Speech (TTS) module

- Built on the Coqui-TTS library (TTS), synthesising Chhattisgarhi text strictly in Devanagari script.
- Integrates speed-pacing control (length_scale) at the model level to adjust response pacing dynamically.

29.3 Models used

Component	Detail
Primary Chhattisgarhi ASR	Meta's Massively Multilingual Speech model (facebook/mms-1b-all), using the Wav2Vec2-CTC backbone with the 'hne' (Chhattisgarhi) language adapter, selected because OpenAI Whisper does not natively support Chhattisgarhi
English/fallback ASR	faster-whisper (CTranslate2), e.g. small.en, retained as a fallback when the language flag is switched
Primary TTS synthesiser	VITS (Variational Inference with adversarial learning for end-to-end Text-to-Speech) architecture
TTS checkpoints	Customised, pre-trained best_model.pth checkpoints for both male and female Chhattisgarhi voices

29.4 Measured and qualitative observations to date

- STT inference speed: the MMS model used is a 1-billion-parameter model, substantially heavier than typical edge STT models; a benchmarking script (`test_mms_speed.py`) is in place to evaluate inference time against audio length to determine whether live partial transcripts are sustainable on target CPUs — this evaluation is ongoing and no final figure is claimed here.
- TTS real-time factor: local notebook execution (`indictts.ipynb`) of the VITS model on GPU shows a processing time of approximately 117 ms and a real-time factor (RTF) of approximately 0.0138 for a standard sentence — i.e., synthesis completing roughly 72× faster than the resulting audio's playback duration. This is a local benchmark observation, not a formally published result.
- Word Error Rate (WER): no calibrated WER benchmark on a standard test set has yet been produced for this pipeline; this is explicitly one of the evaluation tasks planned for Phase 6 of this Minor Project (Section 24).
- Devanagari CTC correction: the MMS model's raw CTC output occasionally inserts stray spaces before Devanagari matras, producing detached characters/dotted circles; a regex post-processor (`clean_mms_devanagari`) was built to correct this and materially improves output readability.
- Domain vocabulary biasing: the pipeline supports initial-prompt biasing to reduce misrecognition of domain-specific proper nouns, previously applied in a different (clinical/appointment) context; this same mechanism will be repurposed for agricultural product and crop-name vocabulary in this Minor Project.
- Barge-in handling: the pipeline emits a `speech_started` event carrying an `interrupt_previous_response` flag, allowing playback to be halted immediately when the user begins speaking again, supporting a more natural conversational flow.

29.5 What this preliminary work does and does not establish

This prior work demonstrates that real-time Chhattisgarhi ASR and TTS are achievable locally, on student hardware, using entirely open-source models — the single technical claim this Minor Project relies on most heavily from its preliminary work. It does not include any agricultural vocabulary tuning, e-commerce logic, translation stage, LLM tool-calling, safety-boundary design, or formal accuracy evaluation, all of which are new contributions of this Minor Project and are scoped and evaluated as such throughout this document.

30. Institutional Requirements and Support

The project currently has no allocated budget and no confirmed institutional sponsorship of any specific external service. The following items are noted as either already available or as requiring confirmation, without assuming approval that has not been granted.

- Speech-lab facilities exist at the institute; at present, the team is proceeding on student-owned hardware, and any additional lab access, if made available, would be used to accelerate model fine-tuning and evaluation in Phase 3 and Phase 6.
- Bhashini API access: registration is free and self-service through the government ULCA portal; the team has not yet completed registration, and any usage terms, rate limits, or eligibility conditions for academic/student use are to be confirmed directly with Bhashini during Phase 1 rather than assumed in advance.
- Any paid LLM API, cloud GPU, or hosting sponsorship: not currently available; would be used only if formally approved by the institute, and is treated throughout this proposal as a contingent future possibility, not a dependency of the MVP.
- Supervision: Prof. Santosh Kumar (Associate Professor, CSE) supervises the project; no additional co-supervisor or institutional collaborator is proposed at this stage.

Where institutional support for any resource is uncertain, this proposal marks it 'to be confirmed' or 'subject to institutional approval' rather than presenting it as secured, consistent with the guidance under which this proposal was prepared.

31. Conclusion

This proposal describes a scoped, technically grounded B.Tech Minor Project: a Chhattisgarhi voice-first agricultural product-discovery and cart-management prototype, built primarily on open-source components the team has already partially demonstrated, and constrained by an explicit, tested safety boundary that keeps the AI in a retail-assistant role rather than an agronomic-advisor role. The project deliberately separates what has already been demonstrated (Level 1), what will be built and evaluated over the next sixteen weeks (Level 2, the MVP), and what might be pursued afterward as a startup-oriented product direction (Level 3), so that the academic deliverable is never inflated beyond what three students can responsibly build, test, and defend on a zero budget.

The team considers the project feasible specifically because its scope has been deliberately narrowed — away from payments, live vendor logistics, and open-ended agronomic advisory AI — toward a well-instrumented voice pipeline and a safety-bounded conversational shopping agent that

can be measured honestly and reported transparently, consistent with the evaluation methodology set out in this document.

References

- ARTPARK & IISc SPIRE Lab (2026). SraVaani-1.0: Scaling Inclusive Speech Recognition for Indic Languages. arXiv:2608.08235. Model card: huggingface.co/ARTPARK-IISc/SraVaani-1.0.
- Bhatt, A. et al. (2023/2026). RESPIN-S1.0: A read speech corpus of 10,000+ hours in dialects of nine Indian Languages. IISc SPIRE Lab / NeurIPS Datasets and Benchmarks. Dataset: spiredatasets.ee.iisc.ac.in/respincorpus.
- IISc SPIRE Lab / SYSPIN Project (GIZ, Germany). SYSPIN_S1.0 Corpus — TTS datasets for nine Indian languages including Chhattisgarhi. sypin.iisc.ac.in / spiredatasets.ee.iisc.ac.in.
- NLLB Team, Meta AI (2022). No Language Left Behind: Scaling Human-Centered Machine Translation. Chhattisgarhi supported under FLORES-200 code hne_Deva. github.com/facebookresearch/flores (NLLB-Seed); huggingface.co/facebook/nllb-200-distilled-600M.
- Pratap, V. et al., Meta AI (2023). Scaling Speech Technology to 1,000+ Languages (Massively Multilingual Speech, MMS). huggingface.co/facebook/mms-1b-all.
- AI4Bharat. IndicTrans2 and the Indic NLP Catalog — coverage of India's 22 Eighth-Schedule scheduled languages. ai4bharat.github.io/indicnlp_catalog.
- Digital India Bhashini Division, Ministry of Electronics and Information Technology, Government of India. Bhashini / ULCA API documentation and onboarding. bhashini.gitbook.io/bhashini-apis; bhashini.gov.in/ulca.
- Indira Gandhi Krishi Vishwavidyalaya (IGKV), Raipur, Chhattisgarh — to be consulted directly during Phase 1 for verified crop-variety and seed-rate reference data; no specific figures are cited from IGKV in this proposal pending direct verification.
- Wikipedia contributors. Chhattisgarhi language. en.wikipedia.org/wiki/Chhattisgarhi_language — used only for general demographic/linguistic background (approx. 16–16.25 million native speakers, Eastern Hindi Indo-Aryan classification, Devanagari script).
- Coqui-TTS (VITS architecture) and Silero VAD — open-source components underlying the team's Level 1 preliminary speech-to-speech prototype, documented in Section 30.